

```
struct rte_ip_frag_tbl
*g_frag_tbl[FRAG_NUM_PORTS] = { NULL, NULL };
struct rte_ip_frag_death_row
g_frag_death_row[FRAG_NUM_PORTS] = {
{ .cnt = 0 }, { .cnt = 0 }
};
```

/* — NEW: inner IP (pre-tunneling) tables — */

```
struct rte_ip_frag_tbl
*g_inner_frag_tbl[FRAG_NUM_PORTS] = { NULL,
NULL };
struct rte_ip_frag_death_row
g_inner_frag_death_row[FRAG_NUM_PORTS] = {
{ .cnt = 0 }, { .cnt = 0 }
};
```

```
int ip_frag_reasm_init(void)
{
uint64_t ttl_cycles = rte_get_tsc_hz() *
(uint64_t)FRAG_TBL_TTL_SEC;
```

```
int sock = rte_socket_id();
```

```
int p;
```

```
for (p = 0; p < FRAG_NUM_PORTS; p++) {
```

```
g_frag_tbl[p] = rte_ip_frag_table_create(
```

```
FRAG_TBL_NUM_BUCKETS,
```

```
FRAG_TBL_BUCKET_ENTRIES,
```

```
FRAG_TBL_MAX_ENTRIES,
```

```
ttl_cycles,
```

```
sock
```

```
);
```

```
if (g_frag_tbl[p] == NULL) {
```

```
printf("ERROR: outer table_create() failed for
```

```
port %d\n", p);
```

```
return -1;
```

```
}
```

```
/* NEW: allocate inner table with identical
```

```
parameters */
```

```
g_inner_frag_tbl[p] = rte_ip_frag_table_create(
```

```
FRAG_TBL_NUM_BUCKETS,  
FRAG_TBL_BUCKET_ENTRIES,  
FRAG_TBL_MAX_ENTRIES,  
ttl_cycles,  
sock  
);  
if (g_inner_frag_tbl[p] == NULL) {  
    printf("ERROR: inner table_create() failed for port  
%d\n", p);  
    return -1;  
}  
}  
  
return 0;  
}  
  
void ip_frag_reasm_drain_deathrow(void)  
{  
    int p;  
    for (p = 0; p < FRAG_NUM_PORTS; p++) {
```

```
if (g_frag_death_row[p].cnt != 0)
rte_ip_frag_free_death_row(&g_frag_death_row[p
],
FRAG_PREFETCH_OFFSET);

/* NEW: drain inner death row too */
if (g_inner_frag_death_row[p].cnt != 0)
rte_ip_frag_free_death_row(&g_inner_frag_death_
row[p],
FRAG_PREFETCH_OFFSET);
}
}
```

```
extern struct rte_ip_frag_tbl
*g_inner_frag_tbl[FRAG_NUM_PORTS];
extern struct rte_ip_frag_death_row
g_inner_frag_death_row[FRAG_NUM_PORTS];
```

